Lock Management API

Setup, authenticated endpoints, realtime alerts, geofences, RFID enforcement, lock configuration, retention, and TCP behavior.

Secure NestJS API baseline for a smart lock management platform with authentication, lock records, RFID card commands, alert storage, and JT701D TCP ingestion.

Stack

- NestJS 11
- PostgreSQL with PostGIS ready for later lock location/device data
- TypeORM
- JWT authentication with Passport
- bcrypt password hashing
- Helmet, CORS, validation pipes, and request throttling
- JT701D TCP listener for lock messages and RFID command responses

Local Setup

```
npm install
npm run start:dev
```

Create a `.env` file before starting. For Neon, use the connection string directly:

```
NODE_ENV=development
PORT=3000
CORS_ORIGIN=http://localhost:3001,http://localhost:3000

DATABASE_URL=postgresql://USER:PASSWORD@HOST.neon.tech/DBNAME?sslmode=require
DB_SSL=true
DB_SYNCHRONIZE=true

JWT_SECRET=your-long-random-secret-at-least-32-characters
JWT_EXPIRES_IN=15m
DEVICE_CONFIG_ENCRYPTION_KEY=another-random-secret-at-least-32-characters

TCP_HOST=0.0.0.0
TCP_PORT=8989
TCP_COMMAND_TIMEOUT_MS=60000

RETENTION_ENABLED=true
DATA_RETENTION_DAYS=30
RETENTION_ARCHIVE_BATCH_SIZE=1000
ARCHIVE_DATABASE_URL=postgresql://USER:PASSWORD@ARCHIVE_HOST.neon.tech/ARCHIVE_DB?sslmode=require
ARCHIVE_DB_SSL=true
```

Set `DB_SYNCHRONIZE=true` only for first local prototyping/table creation. Keep it `false` after the schema exists and for shared/staging/production databases. If using Neon, run this once in the Neon SQL Editor:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

`DEVICE_CONFIG_ENCRYPTION_KEY` encrypts SIM APN passwords with AES-256-GCM before they are saved. Use a different secret from `JWT_SECRET`, keep it stable between deployments, and do not rotate it without re-encrypting existing APN passwords. When `DB_SYNCHRONIZE=false`, run `database/lock-configurations.sql` once in

the Neon SQL Editor.

Retention and archive behavior:

- The live database keeps high-volume operational data for `DATA_RETENTION_DAYS` days. Default: `30`.
- The daily retention job runs at `02:15 UTC`.
- It archives the oldest expired day first, then soft-deletes those live rows with `deletedAt`.
- Example: with 30 days retention, when July 1 is reached, June 1 becomes eligible. The next day, June 2 becomes eligible, and so on.
- Archived rows are copied into the separate database from `ARCHIVE_DATABASE_URL` before soft delete.
- The archive database stores full original rows as JSON in `archived_records`, with `source_table`, `source_id`, `terminal_id`, `data_timestamp`, and `archived_at`.
- Currently archived high-volume tables: `lock_events`, `lock_positions`, and `geofence_transitions`.
- Set `DATA_RETENTION_DAYS` per client contract, for example `30`, `60`, or `90`.

API

All routes are prefixed with `/api`.

Public routes:

- `GET /api`
- `POST /api/auth/register`
- `POST /api/auth/login`

Protected lock-management routes require `Authorization: Bearer <accessToken>`:

- `GET /api/locks`
- `POST /api/locks`
- `GET /api/locks/:terminalId`
- `PATCH /api/locks/:terminalId`
- `GET /api/locks/:terminalId/configuration`
- `PATCH /api/locks/:terminalId/configuration`
- `GET /api/locks/:terminalId/rfid-cards`
- `POST /api/locks/:terminalId/rfid-cards`
- `POST /api/locks/:terminalId/rfid-cards/sync`
- `PUT /api/locks/:terminalId/rfid-cards/:cardNumber/role`
- `DELETE /api/locks/:terminalId/rfid-cards`
- `DELETE /api/locks/:terminalId/rfid-cards/all`
- `GET /api/locks/:terminalId/rfid-cards/groups/:group`
- `GET /api/alerts`
- `PATCH /api/alerts/:id/status`
- `GET /api/locks/:terminalId/events`
- `POST /api/locks/:terminalId/events`
- `GET /api/devices`
- `GET /api/history/:terminalId`

- `GET /api/dashboard/summary`
- `GET /api/reports`
- `GET /api/reports/alerts`
- `GET /api/reports/geofences`
- `GET /api/reports/unlocks`
- `GET /api/reports/mileage`
- `GET /api/reports/battery`
- `GET /api/geo-boundaries`
- `GET /api/geo-boundaries/:id`
- `GET /api/alerts/stream`
- `POST /api/unlock`
- `POST /api/clearcache`
- `POST /api/restart`
- `POST /api/battery/threshold/set`
- `GET /api/battery/threshold/query/:terminalId`
- `POST /api/password/modify`
- `GET /api/password/query/:terminalId`
- `POST /api/deepsleep/set`
- `GET /api/deepsleep/query/:terminalId`
- `POST /api/vip/phone/set`
- `GET /api/vip/phone`
- `DELETE /api/vip/phone`
- `GET /api/vip/phone/query/:terminalId/:index`
- `POST /api/vip/sms/set`
- `GET /api/vip/sms/query/:terminalId`
- `GET /api/geofences`
- `POST /api/geofences`
- `POST /api/geofences/from-boundary`
- `PATCH /api/geofences/:id`
- `DELETE /api/geofences/:id`
- `POST /api/locks/:terminalId/access/check`
- `GET /api/geofence/device/query/:terminalId`
- `POST /api/geofence/device/set`

Optimized list filters:

- `GET /api/locks`: `search`, `status=online|offline|unknown`, `page`, `limit=1-200`.
- `GET /api/devices`: `search`, `isPositioned=true|false`, `limit=1-1000`.
- `GET /api/alerts` and `GET /api/locks/:terminalId/events`: `terminalId`, `status`, `type`, `severity`, `from`, `to`, `page`, `limit=1-200`.

- `GET /api/locks/:terminalId/rfid-cards`: `search`, `role`, `syncStatus`, `installedOnLock`, `limit=1-100`.
- `GET /api/geofences`: `search`, `terminalId`, `shapeType`, `accessMode`, `assigned`, `page`, `limit=1-200`.
- `GET /api/geo-boundaries`: `type`, `search`, `countryCode`, `continent`, `page`, `limit=1-100`, `includeMetadata`.

List endpoints remain arrays for frontend compatibility. Pagination limits the returned slice without changing the response shape.

RFID card commands follow JT701D `P41`:

- Add cards: `(P41,1,1,count,cards...)`
- Delete cards: `(P41,1,2,count,cards...)`
- Clear cards: `(P41,1,3)`
- Query group: `(P41,0,group)`

Querying an RFID group also syncs returned cards into the local database, so cards added directly on the physical lock can appear in the API.

Static password commands follow JT701D `P44`:

- Query current static password: `(P44,1)` through `GET /api/password/query/:terminalId`.
- Modify static password: `(P44,newPassword,currentPassword)` through `POST /api/password/modify`.

Example password modify request:

```
{
  "terminalId": "8034400004",
  "currentPassword": "888888",
  "newPassword": "123456"
}
```

VIP phone query:

When adding a VIP phone with `POST /api/vip/phone/set`, the backend also enables SMS alarm delivery for that slot by querying `P12,0` and then sending `P12,1,...` with the matching VIP flag set to `1`.

```
GET /api/vip/phone?terminalId=8034400004
Authorization: Bearer <accessToken>
```

Returns all five VIP phone slots by sending `P11` queries sequentially:

```
{
  "success": true,
  "terminalId": "8034400004",
  "phones": [
    { "success": true, "index": 1, "phoneNumber": "+212600000001" },
    { "success": true, "index": 2, "phoneNumber": "" }
  ]
}
```

To query one slot:

```
GET /api/vip/phone?terminalId=8034400004&index=3
```

To delete one VIP phone slot:

```
DELETE /api/vip/phone?terminalId=8034400004&index=3
Authorization: Bearer <accessToken>
```

This clears the slot on the physical lock by sending:

```
(P11,1,3,)
```

It also disables SMS alarm delivery for that slot with `P12`.

Lock Configuration Endpoints

Both endpoints require:

- Path parameter `terminalId`: the lock terminal ID already stored by the API, for example `8034400004`.
- Header `Authorization: Bearer <accessToken>`.
- The lock must exist in `lock_devices`; otherwise the API returns `404`.

The API never returns an APN password. If the lock returns an APN password during a physical `P06` query, the API encrypts and stores it, then returns `apnPasswordConfigured` to indicate whether a password is configured.

GET /api/locks/:terminalId/configuration

Loads the configuration saved by the backend. If the lock is connected over TCP, the API first queries the physical lock for SIM settings with `P06,0` for SIM1 and `P06,2` for SIM2, saves the returned IP address, port, APN, and APN user, then returns the refreshed configuration.

Request:

```
GET /api/locks/8034400004/configuration
Authorization: Bearer <accessToken>
```

No request body or query parameters are expected.

Response when the lock has no saved configuration:

```
{
  "terminalId": "8034400004",
  "configured": false,
  "sim1": null,
  "sim2": null,
  "trackingUploadIntervalSeconds": null,
  "wakeUpIntervalMinutes": null,
  "vibrationLevelMg": null,
  "sync": {
    "sim1": null,
    "sim2": null,
    "reporting": null,
    "vibration": null
  }
}
```

Response when a configuration exists:

```
{
  "terminalId": "8034400004",
  "configured": true,
  "sim1": {
    "ipAddress": "jt701.jointcontrols.com",
    "port": 10001,
    "apn": "CMIOT",
    "apnUser": "",
    "apnPasswordConfigured": true
  },
  "sim2": {
    "ipAddress": "120.24.26.10",
    "port": 10001,
    "apn": "internet",
    "apnUser": "",
    "apnPasswordConfigured": false
  },
  "trackingUploadIntervalSeconds": 30,
  "wakeUpIntervalMinutes": 30,
  "vibrationLevelMg": 126,
  "sync": {
    "sim1": {
      "status": "synced",
      "error": null,
      "syncedAt": "2026-06-24T14:00:00.000Z"
    },
    "sim2": {
      "status": "pending",
```

```
        "error": null,
        "syncedAt": null
      },
      "reporting": {
        "status": "synced",
        "error": null,
        "syncedAt": "2026-06-24T14:00:00.000Z"
      },
      "vibration": {
        "status": "failed",
        "error": "Lock did not answer P37 command within 60s",
        "syncedAt": null
      }
    },
    "createdAt": "2026-06-24T13:59:55.000Z",
    "updatedAt": "2026-06-24T14:00:00.000Z"
  }
```


Response field meanings:

- `configured`: `false` until the first valid PATCH creates the lock configuration record.
- `sim1` and `sim2`: saved connection and APN settings, or `null` when that SIM profile has not been configured.
- `trackingUploadIntervalSeconds`: how often the awake lock uploads data through JT701D `P04`.
- `wakeUpIntervalMinutes`: RTC wake-up/reporting interval through JT701D `P04`.
- `vibrationLevelMg`: motion detection sensitivity through JT701D `P37`; lower non-zero values are more sensitive.
- `sync.sim1` and `sync.sim2`: physical synchronization state of the corresponding `P06` command.
- `sync.reporting`: physical synchronization state of the combined `P04` interval command.
- `sync.vibration`: physical synchronization state of the `P37` command.
- `status: "synced"`: the physical lock acknowledged the latest desired values.
- `status: "pending"`: values are saved, but the lock was offline or disconnected before synchronization completed. The backend retries when it reconnects.
- `status: "failed"`: the command timed out or failed while the lock was connected. The desired values remain saved for retry.
- `error`: failure description when status is `failed`; otherwise normally `null`.
- `syncedAt`: timestamp of the last successful synchronization. A later pending/failed change does not erase the previous successful timestamp.

PATCH /api/locks/:terminalId/configuration

Creates or partially updates the saved configuration. Send only the fields that should change. Omitted fields keep their existing values.

Accepted request body:


```
{
  "sim1": {
    "ipAddress": "string",
    "port": 10001,
    "apn": "string",
    "apnUser": "string",
    "apnPassword": "string"
  },
  "sim2": {
    "ipAddress": "string",
    "port": 10001,
    "apn": "string",
    "apnUser": "string",
    "apnPassword": "string"
  },
  "trackingUploadIntervalSeconds": 30,
  "wakeUpIntervalMinutes": 30,
  "vibrationLevelMg": 126
```

```
}
```

Every top-level field is optional, but the body must contain at least one configuration value:

- `sim1` and `sim2`: optional SIM configuration objects.
- `ipAddress`: required the first time that SIM is configured; IP address or hostname, maximum 255 characters, without spaces, commas, or parentheses.
- `port`: required the first time that SIM is configured; integer from `1` to `65530`.
- `apn`: required the first time that SIM is configured; non-empty string up to 50 characters.
- `apnUser`: optional string up to 50 characters. Send `""` to clear it.
- `apnPassword`: optional string up to 50 characters. Send `""` to clear it. Omit it to preserve the current encrypted password.
- `trackingUploadIntervalSeconds`: optional integer from `5` to `600`.
- `wakeUpIntervalMinutes`: optional integer from `5` to `1440`.
- `vibrationLevelMg`: optional integer. Use `0` to disable motion detection or a value from `63` to `500`.
- APN fields cannot contain commas or parentheses because they are encoded into comma-separated JT701D commands.

Complete first-time request:

```
PATCH /api/locks/8034400004/configuration
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "sim1": {
    "ipAddress": "jt701.jointcontrols.com",
    "port": 10001,
    "apn": "CMIOT",
    "apnUser": "",
    "apnPassword": "sim-one-secret"
  },
  "sim2": {
    "ipAddress": "120.24.26.10",
    "port": 10001,
    "apn": "internet",
    "apnUser": "",
    "apnPassword": ""
  },
  "trackingUploadIntervalSeconds": 30,
  "wakeUpIntervalMinutes": 30,
  "vibrationLevelMg": 126
}
```

Partial update examples:

```
{
  "trackingUploadIntervalSeconds": 60,
  "wakeUpIntervalMinutes": 45
}

{
  "sim2": {
    "apn": "new-apn",
    "apnPassword": "new-secret"
  }
}
```

The second SIM example is valid only after SIM2 already has `ipAddress`, `port`, and `apn`. On the first SIM2 update, all three required fields must be sent.

Successful PATCH response:

- Returns HTTP `200` with the same structure as the configured GET response.

- Online acknowledged sections return `status: "synced"`.

- Offline sections return `status: "pending"`; this is still a successful saved update.

- Connected command failures return `status: "failed"` with `error`; the saved desired configuration is not discarded.

- If the patch does not change any saved value, no JT701D command is sent and the current configuration is returned.

Physical command behavior:

- Changed reporting values send one combined `P04` command.

- Changed vibration sends `P37`.

- Changed SIM2 sends `P06,3`.

- Changed SIM1 sends `P06,1`.

- Commands are sent in this order: reporting, vibration, SIM2, SIM1. IP-changing commands are last because changing the destination may disconnect the current TCP session.

Common error responses:

```
{
  "statusCode": 400,
  "message": "SIM1 configuration requires ipAddress, port, and apn",
  "error": "Bad Request"
}

{
  "statusCode": 400,
  "message": ["trackingUploadIntervalSeconds must not be less than 5"],
  "error": "Bad Request"
}

{
  "statusCode": 401,
  "message": "Unauthorized"
}

{
  "statusCode": 404,
  "message": "Lock device not found",
  "error": "Not Found"
}
```

Reports

All report routes require `Authorization: Bearer <accessToken>`.

Common filters are `from`, `to`, `terminalId`, `groupBy=day|week|month`, `page`, and `limit`. The default range is the last 30 days, the default grouping is `day`, and `limit` can be `1-100`.

Every response contains `range`, `filters`, a report-specific `summary`, chart-ready `timeline`, `pagination`, and detailed `rows`.

- `GET /api/reports/alerts`

- Extra filters: `type`, `severity`, `status`. - Returns alert totals, unresolved/critical counts, grouped counts, timeline, and alert rows.

- `GET /api/reports/geofences`

- Extra filter: `geofenceId`. - Returns geofence entries, exits, unlocks inside sites, affected locks, configuration/rule rows, and timeline.

- `GET /api/reports/unlocks`

- Extra filters: `geofenceId`, `method`. - Methods: `rfid`, `static_password`, `dynamic_password`, `bluetooth`, `other`. - Returns total openings, openings inside sites, card/password totals, method groups, timeline, and unlock rows. - JT701D P45 RFID/dynamic verification values `1-10` and `98` count as successful unlocks. Value `99` is a

geofence rejection and `0` is a failed verification.

- `GET /api/reports/mileage`

- Returns total kilometers and per-lock starting mileage, ending mileage, calculated distance, and timestamps.

- `GET /api/reports/battery`

- Extra filter: `below=0-100`. - Returns average/minimum/maximum battery, low/charging samples, timeline, and per-lock battery rows.

Example:

```
GET /api/reports/unlocks?from=2026-06-01T00:00:00.000Z&to=2026-06-24T23:59:59.999Z&terminalId=8034400004&method=rfid&groupBy
Authorization: Bearer <accessToken>
```

Expected response structure:

```
{
  "range": {
    "from": "2026-06-01T00:00:00.000Z",
    "to": "2026-06-24T23:59:59.999Z",
    "groupBy": "day"
  },
  "filters": {
    "terminalId": "8034400004"
  },
  "summary": {
    "totalOpened": 18,
    "openedInsideSite": 12,
    "openedByCard": 18,
    "openedByPassword": 0,
    "affectedLocks": 1,
    "byMethod": [{ "key": "rfid", "count": 18 }]
  },
  "timeline": [],
  "pagination": {
    "page": 1,
    "limit": 25,
    "total": 18,
    "pages": 1
  },
  "rows": []
}
```

TCP positions now persist geofence entry/exit transitions, and lock events store the geofences containing their coordinates. Reports use the live retained database; with the default policy, detailed data covers 30 days. Older archive data is not merged into report responses.

Dashboard summary:

- `GET /api/dashboard/summary` returns KPI cards and chart-ready data for the dashboard.

- Optional query params: `from`, `to`, and `terminalId`.

- Date range filtering applies to position-derived KPIs, lock activity, alarm count, top alarms, RFID usage, and trip heatmap rows.

- `terminalId` scopes every dashboard metric, including lock status and RFID synchronization.

- `totalAssets`, `online`, `offline`, and `connectionStatus` are current snapshot values.

- The response includes `kpis`, `connectionStatus`, `lockActivity`, `topAlarms`, `lockStateDistribution`, `rfidSyncStatus`, `topRfidCards`, `tripHeatmap`, and `heatMapTracks`.

- `lockActivity.summary` contains alert, stopped/locked, and unlocked counts for the period; `lockActivity.ranking` gives the most common event types.

- `topRfidCards` gives the most-used card numbers with label/role when known.

- `tripHeatmap` groups lock/unlock activity by the geofence name stored on each event, with `Outside geofences` as the fallback.

- `heatMapTracks` is the frontend-friendly heatmap list. Each item includes compatible aliases: `location/value`, `city/count`, `place/activity`, and `name/events`.

Example:

```
GET /api/dashboard/summary?terminalId=8034400004&from=2026-06-01T00:00:00.000Z&to=2026-06-22T23:59:59.999Z
```

Dashboard demo data:

- Run `npm run seed:dashboard` to create dynamic demo data for the dashboard in the configured database.
- The seed creates demo locks `DEMOLOCK001` through `DEMOLOCK008`, 30 days of positions, lock/unlock events, alerts, RFID cards, and demo geofences.
- Rerunning the command refreshes only this demo dataset; it deletes and recreates records owned by the `DEMOLOCK` terminal IDs and geofences named `Demo ...`.
- The frontend can test global metrics with `GET /api/dashboard/summary` or one lock with `GET /api/dashboard/summary?terminalId=DEMOLOCK001`.

Realtime alerts:

- `GET /api/alerts` returns the latest stored alerts for initial loading and fallback polling.
- Filter alerts with `terminalId`, `status`, `type`, `severity`, `from`, `to`, `page`, and/or `limit`: `GET /api/alerts?terminalId=8034400004&status=unread&severity=critical&from=2026-06-01T00:00:00.000Z&to=2026-06-23T23:59:59.999Z&page=1&limit=50`.
- Alert status defaults to `unread`. Valid status values are `unread`, `read`, `investigating`, and `resolve`.
- Update alert status with `PATCH /api/alerts/:id/status`.
- `GET /api/alerts/stream` opens a Server-Sent Events stream for new alerts.
- Add `terminalId` to listen to one lock only: `GET /api/alerts/stream?terminalId=8034400004`.
- Stream alert events use event name `alert`; keepalive events use event name `keepalive`.
- Use `fetch` streaming with `Authorization: Bearer <accessToken>` because this keeps the JWT out of the URL.

Frontend example:

```
const token = localStorage.getItem('accessToken');
const params = new URLSearchParams();

params.set('terminalId', '8034400004');

const response = await fetch(`/api/alerts/stream?${params.toString()}`, {
  headers: {
    Authorization: `Bearer ${token}`,
  },
});

if (!response.ok || !response.body) {
  throw new Error('Could not open alert stream');
}

const reader = response.body.getReader();
const decoder = new TextDecoder();
let buffer = '';

while (true) {
  const { done, value } = await reader.read();

  if (done) break;

  buffer += decoder.decode(value, { stream: true });
  const messages = buffer.split('\n\n');
  buffer = messages.pop() ?? '';

  for (const message of messages) {
    const eventName = message
```

```
        .split('\n')
        .find((line) => line.startsWith('event: '))
        ?.slice(7);
      const data = message
        .split('\n')
        .find((line) => line.startsWith('data: '))
        ?.slice(6);

      if (eventName === 'alert' && data) {
        const alert = JSON.parse(data);

        setAlerts((current) => [alert, ...current]);
        showToast(`${alert.severity}: ${alert.type}`);
      }
    }
  }
```

Example alert status update:

```
PATCH /api/alerts/alert-uuid/status
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "status": "investigating"
}
```

Example alert date range filters:

```
GET /api/alerts?from=2026-06-01T00:00:00.000Z&to=2026-06-23T23:59:59.999Z
GET /api/alerts?status=unread&from=2026-06-01T00:00:00.000Z
GET /api/locks/8034400004/events?status=resolve&to=2026-06-23T23:59:59.999Z
```

RFID card roles:

- `admin`: one admin card per lock, allowed anywhere.
- `limited`: up to 19 limited cards per lock, checked against active geofence rules.
- `POST /api/locks/:terminalId/rfid-cards` defaults to `limited` when `role` is omitted.
- Include `role: "admin"` only when intentionally assigning the one admin card.
- Use `PUT /api/locks/:terminalId/rfid-cards/:cardNumber/role` to promote or demote a card.
- Use `POST /api/locks/:terminalId/rfid-cards/sync` to immediately recalculate geofence access from the latest saved lock position and push needed `P41 add/delete` changes.
- Card responses include physical sync state: `installedOnLock`, `lastSyncStatus`, `lastSyncError`, and `lastSyncedAt`.
- Physical geofence enforcement uses JT701D `P41`: blocked limited cards are temporarily removed from the lock whitelist, then restored when allowed again.
- Admin cards are never removed by geofence automation.

Geo boundary catalog:

- Use `geo_boundaries` for reference map data such as country polygons.
- Use `geofences` for real business rules such as whether a lock can open inside/outside a boundary.
- `GET /api/geo-boundaries` searches imported boundaries without returning the heavy geometry.
- `GET /api/geo-boundaries/:id` returns one boundary with GeoJSON geometry for map drawing.
- `POST /api/geofences/from-boundary` creates a real geofence rule linked to a selected boundary.
- Boundary-linked geofences use PostGIS point-in-polygon checks.

Geo boundary search filters:

- `type`: `continent`, `country`, `region`, or `city`.
- `search`: partial boundary name match, for example `morocco` or `costa`.

- `countryCode`: exact ISO country code match, for example `MAR` or `CRI`.
- `continent`: exact case-insensitive continent name match, for example `Africa`.
- `page`: result page, starting at `1`.
- `limit`: number of records to return, from `1` to `100`; defaults to `50`.
- `includeMetadata`: defaults to `false`. Set it to `true` only when list metadata is needed; geometry still comes only from `GET /api/geo-boundaries/:id`.

Import the provided country GeoJSON file:

```
npm run geo:import
```

Or import another GeoJSON FeatureCollection:

```
npm run geo:import -- "C:\path\to\boundaries.geojson"
```

If `DB_SYNCHRONIZE=false`, the import script still creates the `geo_boundaries` table and PostGIS indexes. For `geofences`, add the optional link column manually:

```
ALTER TABLE geofences ADD COLUMN IF NOT EXISTS "geoBoundaryId" uuid;
```

For better database performance on Neon/production, run the SQL in `database/performance-indexes.sql`. It adds indexes for latest lock positions, event history, retention cleanup, RFID card lookups, geofence ordering, and PostGIS boundary checks.

For physical RFID geofence enforcement, run `database/rfid-sync-state.sql` before the performance indexes if `DB_SYNCHRONIZE=false`.

For `/api/devices` position metadata, run `database/position-device-fields.sql` if `DB_SYNCHRONIZE=false`.

For alert status support, run `database/alert-status.sql` if `DB_SYNCHRONIZE=false`.

Example boundary search:

```
GET /api/geo-boundaries?type=country&search=costa&limit=10
GET /api/geo-boundaries?type=country&continent=Africa&limit=100
GET /api/geo-boundaries?type=country&countryCode=MAR
```

Example geofence from imported country boundary:

```
{
  "name": "Costa Rica country rule",
  "terminalIds": ["8034400004"],
  "geoBoundaryId": "boundary-uuid-here",
  "accessMode": "allow_inside",
  "rules": {
    "smsAllowed": true,
    "gprsAllowed": true,
    "rfidAllowed": true,
    "serialAllowed": true,
    "bluetoothAllowed": true,
    "lockAccessAllowed": true
  }
}
```

Geofences can describe three shapes:

- `polygon`: a closed area. Send at least 3 coordinate points.
- `circle`: a center point plus `radiusMeters`.
- `route`: a line/way corridor. Send at least 2 coordinate points plus `radiusMeters` for the corridor width.

Geofence access is controlled by `accessMode`:

- `allow_inside`: limited cards may unlock only when the lock is inside the geofence shape.
- `allow_outside`: limited cards may unlock only when the lock is outside the geofence shape.

`terminalIds` is optional when creating a geofence. When omitted, the geofence is saved with no lock assignment and does not enforce access rules. Assign one or more locks later with `PATCH /api/geofences/:id`. A geofence never restricts an unrelated lock.

Update a geofence's inside/outside logic or checkbox rules:

```
PATCH /api/geofences/geofence-uuid
Authorization: Bearer <accessToken>
Content-Type: application/json

{
  "terminalIds": ["8034400004"],
  "accessMode": "allow_outside",
  "rules": {
    "lockAccessAllowed": false,
    "rfidAllowed": false
  }
}
```

Every PATCH field is optional, but at least one field must be sent. Supported fields are `name`, `terminalIds`, `shapeType`, `coordinates`, `radiusMeters`, `accessMode`, and any subset of `rules`. `terminalIds` must contain at least one existing lock terminal ID. Omitted rule booleans retain their current values. The response is the complete updated geofence object.

The updated rule is enforced immediately from the latest saved positioned GPS point, then again on each new positioned GPS report from the lock. Admin cards continue to bypass geofence restrictions; limited cards are physically removed from or restored to the lock through `P41` according to the new result.

Geofence rules include `lockAccessAllowed`. The frontend checkbox should write this boolean to `rules.lockAccessAllowed`. For limited cards, `allow_outside` geofences block when the lock is inside that geofence, and `allow_inside` geofences allow access when the lock is inside at least one assigned allowed zone. A geofence also blocks inside its shape when `rules.lockAccessAllowed` or `rules.rfidAllowed` is `false`. Admin cards bypass these backend restrictions.

Physical RFID swipe enforcement:

- The API database keeps every card and role as the source of truth.
- When a lock reports GPS, the backend evaluates geofences for limited cards.
- When a geofence is created, updated, deleted, or manually synced, the backend also evaluates the latest saved lock position.
- If limited cards are blocked and installed on the lock, the backend sends `P41 delete` for those cards only.
- If limited cards become allowed again and are not installed on the lock, the backend sends `P41 add`.
- Failed/offline syncs are kept as `pending_add`, `pending_delete`, or `failed` and retried on later GPS reports.
- `P59 rfid=false` is not used for per-card blocking because it disables RFID for admin cards too.

Example polygon geofence:

```
{
  "name": "Warehouse yard",
  "terminalIds": ["8034400004"],
  "shapeType": "polygon",
  "accessMode": "allow_inside",
  "coordinates": [
    { "lat": 33.594, "lng": -7.62 },
    { "lat": 33.596, "lng": -7.62 },
    { "lat": 33.596, "lng": -7.617 }
  ],
  "rules": {
    "smsAllowed": true,
    "gprsAllowed": true,
    "rfidAllowed": true,
    "serialAllowed": true,
    "bluetoothAllowed": true,
    "lockAccessAllowed": true
  }
```

```
}
```

Example circle geofence:

```
{
  "name": "Depot radius",
  "terminalIds": ["8034400004"],
  "shapeType": "circle",
  "accessMode": "allow_inside",
  "coordinates": [{ "lat": 33.594, "lng": -7.62 }],
  "radiusMeters": 250,
  "rules": {
    "smsAllowed": true,
    "gprsAllowed": true,
    "rfidAllowed": true,
    "serialAllowed": true,
    "bluetoothAllowed": true,
    "lockAccessAllowed": true
  }
}
```

Example route geofence:

```
{
  "name": "Approved route",
  "terminalIds": ["8034400004"],
  "shapeType": "route",
  "accessMode": "allow_outside",
  "coordinates": [
    { "lat": 33.594, "lng": -7.62 },
    { "lat": 33.6, "lng": -7.61 }
  ],
  "radiusMeters": 100,
  "rules": {
    "smsAllowed": true,
    "gprsAllowed": true,
    "rfidAllowed": true,
    "serialAllowed": true,
    "bluetoothAllowed": true,
    "lockAccessAllowed": true
  }
}
```

The app also starts a TCP listener on `TCP_HOST:TCP_PORT` (`0.0.0.0:8989` by default), matching the reference prototype. RFID card routes send the P41 command to the connected lock and wait up to `TCP_COMMAND_TIMEOUT_MS` for the P41 response.

Incoming JT701D TCP handling:

- Binary `$` frames are buffered, parsed, ACKed with `P69`, and alarm bits are stored as lock events.
- ASCII `P45` lock/unlock reports are parsed, ACKed with `P69`, and stored as lock events.
- ASCII `P22` time sync requests are answered.
- ASCII `P04`, `P06`, `P37`, `P41`, `P43`, `P59`, `P61`, `P44`, `P03`, `P11`, `P12`, and `P15` responses resolve pending API requests.
- GPS reports are checked only against geofences assigned to that lock through `terminalIds`. Active overlapping assigned rules are merged with the most restrictive permissions and sent to the lock with `P59` when channel settings change.

`GET /api/devices` returns each latest positioned lock as:

```
{
  "id": "8034400004",
  "position": {
    "lat": 33.594,
    "lng": -7.62,
    "speed": 0,
    "timestamp": 1719060000000,
    "gpsTimestamp": 1719060000000,
    "battery": "80%",
    "isCharging": false,
    "isLocked": true,
```

```
      "is_positioned": true,
      "mileage": 12345
    }
  }
```

`GET /api/history/:terminalId` returns route points for a lock. By default it returns today's UTC route. The frontend can request a date range with `from` and `to` ISO timestamps and control the maximum returned map points with `maxPoints` (`100-10000`, default `2000`):

```
GET /api/history/8034400004?from=2026-06-01T00:00:00.000Z&to=2026-06-23T23:59:59.999Z&maxPoints=2000
```

Long routes are evenly sampled in chronological order while preserving the first and last points. The query reads only latitude and longitude and excludes soft-deleted or invalid/unpositioned GPS rows.